Project 10 Documentation

Nolan DeSchryver & Matthew Frey

Overview:

This project demonstrates advanced texture mapping techniques in OpenGL, featuring environment, parallax, and bump mapping applied to various 3D objects. The scene includes a checkerboard floor and three 3D models, cylinder, cube, and sphere. The sphere uses environment mapping, the cube uses parallax mapping, and the cylinder uses bump mapping.

Theoretical Background:

Bump Mapping:

Bump mapping changes how light interacts with a surface to make it appear rough or detailed. It does this by adjusting the surface normals, which affects lighting but does not change the actual shape of the object.

Parallax Mapping:

Parallax mapping goes a step further by shifting the texture coordinates based on the viewer's angle and a height map. This creates the illusion that parts of the surface are raised or recessed, making a flat object look like it has real depth.

Environment Mapping (Cube Mapping):

Environment mapping simulates reflections by surrounding the scene with a cube of images, which is called a "cubemap." The object samples this "cubemap" based on the viewing direction, making it appear reflective, like a mirror.

Tangent Space:

Both bump mapping and parallax mapping rely on tangent space. The Tangent-Bitangent-Normal (TBN) matrix transforms vectors into this space so that textures and lighting behave correctly regardless of how the object is rotated.

Phong Lighting:

This project uses the Phong lighting model, which includes:

- Ambient lighting (base light)
- Diffuse lighting (light hitting the surface)
- Specular lighting (reflections and highlights)

Mathematical Concepts

Camera Angles:
The camera's position and direction are used to calculate the view direction vector, which is essential for lighting and advanced shading effects. The view direction is typically computed as:

viewDir = normalize(cameraPos - fragPos)

This vector determines how the surface is viewed and is used in:

- Reflection calculations (environment mapping)
- Parallax mapping (to shift texture coordinates)
- Specular lighting (highlight intensity)

```
front.x = cos(Yaw) * cos(Pitch)
front.y = sin(Pitch)
front.z = sin(Yaw) * cos(Pitch)
Right = normalize(cross(Front, WorldUp))
Up = normalize(cross(Right, Front))
```

Tangent and Bitangent Calculations:

To correctly apply bump and parallax mapping, vectors must be transformed into tangent space using the TBN matrix:

TBN = [T, B, N]

These vectors are calculated per vertex and normalized:

```
vec3 T = normalize(mat3(model) * tangent);
vec3 B = normalize(mat3(model) * bitangent);
vec3 N = normalize(mat3(model) * normal);
mat3 TBN = transpose(mat3(T, B, N));
```

The TBN matrix allows lighting and view vectors to be correctly aligned with the texture surface.

```
e1 = v2 - v1
e2 = v3 - v1
duv1 = uv2 - uv1
duv2 = uv3 - uv1

f = 1.0 / (duv1.x * duv2.y - duv2.x * duv1.y)

tangent = f * (duv2.y * e1 - duv1.y * e2)
bitangent = f * (-duv2.x * e1 + duv1.x * e2)
```

Phong Lighting:

The Phong lighting model combines three components:

I = Ia + Id + Is

Where:

- Ambient: Constant light is applied everywhere
- Diffuse: Based on the angle between light and surface; Id = max(0, N * L)
- Specular: Creates highlights based on reflection; Is = (R * V) ^ n
    - N = normal
    - L = light direction
    - V = view direction
    - R = reflection direction
    - n = shininess factor

These calculations determine how the light interacts with each fragment producing realistic shading.

```
lightDir = normalize(lightPos - FragPos)
viewDir = normalize(viewPos - FragPos)

// Ambient
ambient = ambientStrength * lightColor

// Diffuse (Lambertian)
diffuse = max(dot(normal, lightDir), 0.0) * lightColor

// Specular (Blinn-Phong variant)
reflectDir = reflect(-lightDir, normal)
specular = pow(max(dot(viewDir, reflectDir), 0.0), shininess) * lightColor

finalColor = (ambient + diffuse + specular) * textureColor
```

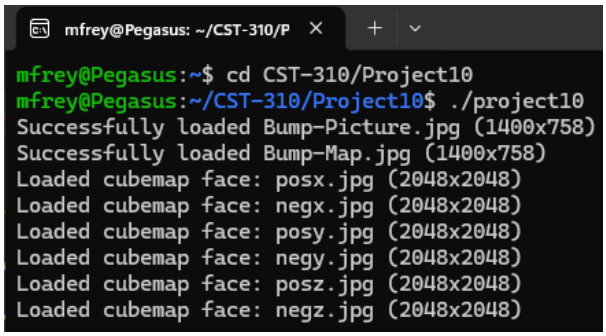
Aesthetic Decisions:

We had to adjust some of the lighting settings for the different shapes because the lighting bounced off the textures different:

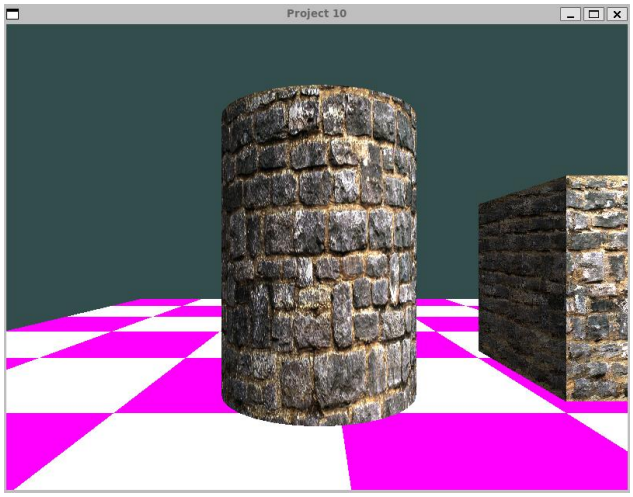
```
// ambient
float ambientStrength = 2; // Set ambient strength
vec3 ambient = ambientStrength * lightColor; // Sets ambient

// diffuse
vec3 lightDir = normalize(lightPos - FragPos); // Sets light direction based on light - frag position
float diff = max(dot(normal, lightDir), 0.0); // Diffuse contribution
vec3 diffuse = diff * lightColor; // Sets diffuse
```

Running and Output:

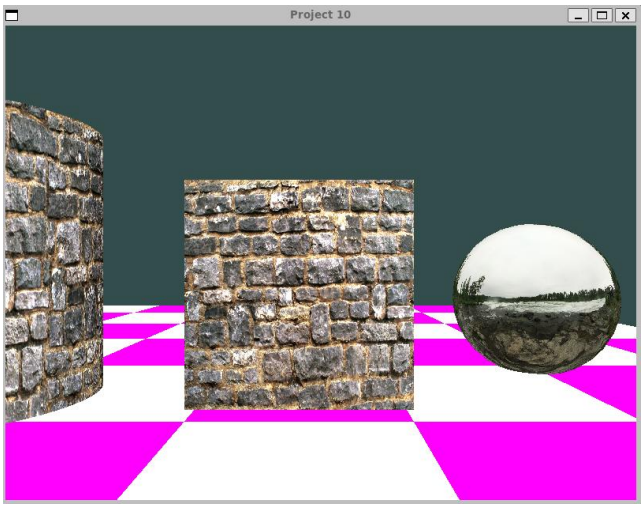
Initializing program:



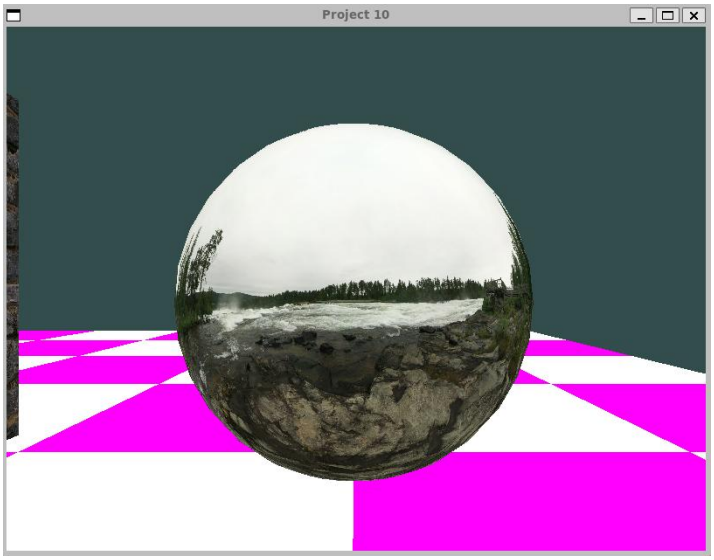
Cylinder (Bump mapping):



Cube (Parallax mapping):



Sphere (Environmental (Cube) mapping):



Video Link:

https://www.loom.com/share/57148f6d11e844119896de0bfe59b41c

References

Guha, S. (2019). *Computer graphics through OpenGL: From theory to experiments* (3rd ed.). Chapman and Hall/CRC.

Khronos Group. (n.d.). *OpenGL reference pages*. https://www.khronos.org/opengl/wiki/

LearnOpenGL. (n.d.-a). *Advanced lighting: Normal mapping*. https://learnopengl.com/Advanced-Lighting/Normal-Mapping

LearnOpenGL. (n.d.-b). *Advanced lighting: Parallax mapping*. https://learnopengl.com/Advanced-Lighting/Parallax-Mapping

LearnOpenGL. (n.d.-c). *Cubemaps*. https://learnopengl.com/Advanced-OpenGL/Cubemaps

Humus. (n.d.). *Cubemap textures*. http://www.humus.name/index.php?page=Textures